Algoritmos golosos - Ejercicios

Contenidos

Contenidos **1**

Ejercicio 2. Ruta Eficiente . . . 2
- Algoritmo Greedy propuesto . . . 2
- Ejemplo de ejecución . . . 3
- Demostración de correctitud . . . 3

Ejercicio 3. Suma subconjuntos . . . 6
- Resolución . . . 6

Ejercicio 5. Planificación de tareas con *deadlines* . . . 8
- Descripción del problema . . . 8
- Restricciones . . . 8
- Objetivo . . . 8
- Especificación formal . . . 8
- Ejemplo . . . 9
- Solución mediante algoritmo Greedy . . . 9
- Pseudocódigo . . . 9
- Ejemplo de ejecución . . . 10
- Demostración de correctitud . . . 10

Ejercicio 6. Producto Escalar . . . 13
- Enunciado del problema . . . 13
- Algoritmo Greedy propuesto . . . 13
- Ejemplo . . . 13
- Demostración de correctitud . . . 14

Ejercicio 2. Ruta Eficiente

Tomás quiere viajar de Buenos Aires a Mar del Plata en su flamante Renault 12. Como está preocupado por la autonomía de su vehículo, se tomó el tiempo de anotar las distintas estaciones de servicio que se encuentran en el camino. Modeló el mismo como un segmento de 0 a M, donde Buenos aires está en el kilómetro 0, Mar del Plata en el M, y las distintas estaciones de servicio están ubicadas en los kilómetros $0 = x_1 \leq x_2 \leq \ldots x_n \leq M$.

Razonablemente, Tomás quiere minimizar la cantidad de paradas para cargar nafta. Él sabe que su auto es capaz de hacer hasta C kilómetros con el tanque lleno, y que al comenzar el viaje este está vacío.

- Proponer un algoritmo *greedy* que indique cuál es la cantidad mínima de paradas para cargar nafta que debe hacer Tomás, y que aparte devuelva el conjunto de estaciones en las que hay que detenerse. Probar su correctitud.
- Dar una implementación de complejidad temporal $O(n)$ del algoritmo.

Algoritmo Greedy propuesto

La estrategia greedy es simple pero efectiva: **siempre avanzar hasta la estación más lejana posible dentro del alcance C kilómetros**.

Descripción intuitiva

Comenzando desde Buenos Aires (o desde la última estación donde cargamos combustible), miramos todas las estaciones que están dentro de nuestro alcance de C kilómetros y elegimos la más lejana. Repetimos este proceso hasta llegar a Mar del Plata.

Pseudocódigo

```
Algoritmo: MinParadas(x[], C, M)
Entrada: x[1..n] = posiciones de estaciones (x[0] = 0 implícito)
C = alcance máximo con tanque lleno
M = distancia total a Mar del Plata
Salida: Conjunto de estaciones donde parar

1.  n <- longitud(x) -1
2.
3.  // Validaciones de casos sin solución
4.  Si x[n] + C < M entonces:
5.      Retornar "No existe solucion"
6.  Fin Si
7.
8.  Para i desde 1 hasta n - 1 hacer:
9.      Si x[i+1] - x[i] > C entonces:
```

```
10.         Retornar "No existe solucion"
11.     Fin Si
12. Fin Para
13.
14. // Inicialización
15. estaciones <- {x[0]}  // Cargamos en la primera estación
16. ultima_est <- 0
17.
18. Mientras x[ultima_est] + C < M hacer:
19.     prox_est <- ultima_est + 1
20.
21.     // Buscar la estación más lejana alcanzable
22.     Mientras prox_est <= n Y x[prox_est] - x[ultima_est] <= C hacer:
23.         prox_est <- prox_est + 1
24.     Fin Mientras
25.
26.     estaciones <- estaciones U {x[prox_est - 1]}
27.     ultima_est <- prox_est - 1
28. Fin Mientras
29.
30. Retornar estaciones
```

Complejidad: $O(n)$, ya que cada estación se examina a lo sumo una vez.

Ejemplo de ejecución

Consideremos un viaje con los siguientes datos:

- Distancia total $M = 400$ km
- Alcance $C = 150$ km
- Estaciones en: $\{0, 80, 140, 200, 280, 350\}$

Iteración	Posición actual	Alcance hasta	Estación elegida
1	0	150	140 (más lejana ≤ 150)
2	140	290	280 (más lejana ≤ 290)
3	280	430	400 (destino alcanzable)

Solución: Paradas en las estaciones ubicadas en km 0, 140, y 280. Total: 3 paradas.

Demostración de correctitud

Planteamiento de la Demostración

Queremos demostrar que la solución S devuelta por el algoritmo es óptima (minimiza la cantidad de paradas).

Definición de las Soluciones

Sean las estaciones elegidas por cada conjunto ordenadas por su posición en la ruta:

- Solución Greedy: $S = \{g_1, g_2, \ldots, g_k\}$
- Solución Óptima: $O = \{o_1, o_2, \ldots, o_m\}$

Queremos probar que $|S| = |O|$, es decir, que $k = m$. Estrategia: Argumento por Intercambio. Demostraremos que podemos transformar la solución óptima O en nuestra solución greedy S, elemento a elemento, sin aumentar el número total de paradas.

El Paso de Intercambio

Supongamos que S y O coinciden en las primeras r estaciones, pero difieren en la siguiente:

$$g_1 = o_1, \quad g_2 = o_2, \quad \ldots \quad g_r = o_r \quad \text{pero} \quad g_{r+1} \neq o_{r+1}$$

- Como el algoritmo avanza buscando maximizar la distancia en cada salto, la estación g_{r+1} es la que tiene la mayor posición $x_{g_{r+1}}$ disponible que cumple con $x_{g_{r+1}} \leq x_{g_r} + C$.
- Dado que $g_r = o_r$, la estación o_{r+1} de la solución óptima también debe cumplir su alcance $x_{o_{r+1}} \leq x_{g_r} + C$, por lo que era una opción válida y evaluada para el algoritmo.

Propiedad Greedy

Por la regla de selección del algoritmo, se garantiza que:

$$x_{g_{r+1}} \geq x_{o_{r+1}}$$

Validez de la Nueva Solución

Definimos una nueva solución modificada O' reemplazando o_{r+1} por g_{r+1}:

$$O' = (O \setminus \{o_{r+1}\}) \cup \{g_{r+1}\}$$

¿Por qué O' sigue siendo una solución válida?

- Por la izquierda: g_{r+1} es alcanzable desde g_r (ya que $g_r = o_r$).
- Por la derecha: Como $x_{g_{r+1}} \geq x_{o_{r+1}}$, el auto avanza lo mismo o más. Esto implica que la distancia faltante hasta la siguiente parada óptima es menor o igual:

$$x_{o_{r+2}} - x_{g_{r+1}} \leq x_{o_{r+2}} - x_{o_{r+1}} \leq C$$

Por lo tanto, g_{r+1} no impide alcanzar la siguiente parada de la secuencia óptima (o_{r+2}). Como O' no presenta saltos infactibles y hemos demostrado que mantiene exactamente el tamaño de O ($|O'| = |O|$), concluimos que O' sigue siendo una solución óptima válida.

Observación importante sobre el reemplazo: ¿Qué sucedería si g_{r+1} ya estuviera presente más adelante en la ruta óptima O (por ejemplo, si $g_{r+1} = o_{r+2}$)? En ese caso, avanzar directo desde g_r hasta g_{r+1} implicaría saltear o_{r+1}, produciendo una ruta válida con estrictamente menos paradas que O. Pero esto es un absurdo, ya que contradice la premisa fundamental de que O es una solución óptima. Por lo tanto, esta situación no puede ocurrir.

Conclusión de la demostración

Aplicando este argumento de forma inductiva, podemos sustituir consecutivamente todos los elementos de la solución óptima hasta que O se transforme en S **sin aumentar el número total de paradas**. Por lo tanto, $|S| = |O|$. Es decir, el algoritmo encuentra el mínimo número de paradas.

Ejercicio 3. Suma subconjuntos

Enunciado:
Tenemos un conjunto de enteros S y un valor t. Queremos encontrar el subconjunto $S' \subseteq S$ más chico tal que $\sum_{s \in S'} s = t$.

A Gervasio le parece obvia la solución: simplemente ir agarrando el número de S más grande que no supere a t hasta llegar a t. Si no se puede llegar a t de esta forma, entonces no hay solución.

Mostrarle a Gervasio un conjunto S y un valor t para el cual esta estrategia no es correcta.

Resolución

Para demostrar que la estrategia greedy propuesta por Gervasio no siempre produce el subconjunto de menor cardinalidad (respetando la definición estricta de conjunto, sin elementos repetidos), es suficiente plantear el siguiente contraejemplo.

Sea el conjunto de enteros $S = \{1, 2, 4, 5, 8, 10\}$ y el valor objetivo $t = 13$.

Ejecución del algoritmo de Gervasio (Greedy):

1. Se busca el número más grande en S que sea ≤ 13. El algoritmo elige el **10**.
2. Se actualiza el valor restante por alcanzar: $13 - 10 = 3$.
3. Se busca el número más grande disponible en $S \setminus \{10\}$ que sea ≤ 3. El algoritmo elige el **2**.
4. Se actualiza el valor restante: $3 - 2 = 1$.
5. Se busca el número más grande disponible en $S \setminus \{10, 2\}$ que sea ≤ 1. El algoritmo elige el **1**.
6. Se actualiza el valor restante: $1 - 1 = 0$. El algoritmo termina correctamente sumando 13.

Bajo la estrategia de Gervasio, se nos propone el subconjunto:

$$S'_{greedy} = \{10, 2, 1\}$$

El tamaño de este subconjunto consta de **3 elementos**.

Solución Óptima:

No obstante, si examinamos el conjunto original, existe una combinación mucho más directa para alcanzar el valor 13. Si optamos por seleccionar el 8 y el 5, obtenemos exactamente:

$$8 + 5 = 13$$

El subconjunto verdaderamente óptimo es:

$$S'_{optimo} = \{8, 5\}$$

el cual consta de únicamente **2 elementos**.

Dado que el subconjunto hallado por el enfoque greedy de Gervasio tiene una mayor cantidad de elementos que la solución óptima ($3 > 2$), queda demostrado que la estrategia de Gervasio **no es correcta**.

Ejercicio 5. Planificación de tareas con deadlines

Descripción del problema

Se tiene un conjunto de n tareas $\mathcal{T} = \{t_1, t_2, \ldots, t_n\}$, donde cada tarea t_i tiene asociado un *deadline* $d_i \in \mathbb{Z}^+$ que representa el tiempo límite antes del cual debe completarse la tarea.

Restricciones

- Cada tarea requiere exactamente **una unidad de tiempo** para completarse.
- Solo se puede ejecutar **una tarea a la vez** (no existe paralelismo).
- El tiempo comienza en $t = 0$ y avanza en unidades discretas: $t \in \{0, 1, 2, \ldots\}$.
- Una tarea t_i se considera completada exitosamente si y solo si termina su ejecución en un tiempo $f_i \leq d_i$, donde f_i es el tiempo de finalización de la tarea.
- Una vez iniciada una tarea, debe completarse sin interrupciones (no hay *preemption*).
- Las tareas pueden ejecutarse en cualquier orden.

Objetivo

Determinar una planificación (orden de ejecución) que **maximice el número total de tareas** que pueden completarse respetando sus respectivos *deadlines*.

Especificación formal

Entrada

- Un entero $n \in \mathbb{Z}^+$ representando el número de tareas.
- Un conjunto de *deadlines* $D = \{d_1, d_2, \ldots, d_n\}$ donde $d_i \in \mathbb{Z}^+$ para todo $i \in \{1, 2, \ldots, n\}$.

Salida

- El número máximo k de tareas que pueden completarse dentro de sus *deadlines*.
- (Opcional) Una secuencia $S = \langle s_1, s_2, \ldots, s_k \rangle$ que representa un orden válido de ejecución, donde $s_j \in \{1, 2, \ldots, n\}$ indica el índice de la j-ésima tarea a ejecutar.

Ejemplo

Considere $n = 4$ tareas con *deadlines* $D = \{2, 1, 3, 2\}$.

Tarea	Deadline
t_1	2
t_2	1
t_3	3
t_4	2

Una solución óptima sería:

- Ejecutar t_2 en el intervalo $[0, 1)$ (se completa en tiempo 1, cumple con $d_2 = 1$)
- Ejecutar t_1 en el intervalo $[1, 2)$ (se completa en tiempo 2, cumple con $d_1 = 2$)
- Ejecutar t_3 en el intervalo $[2, 3)$ (se completa en tiempo 3, cumple con $d_3 = 3$)

Esta planificación completa 3 tareas exitosamente. La tarea t_4 no puede ejecutarse sin violar su *deadline*.

Solución mediante algoritmo Greedy

Para resolver el problema de planificación de tareas con *deadlines*, utilizaremos un algoritmo greedy con la siguiente estrategia:

1. Ordenar todas las tareas por sus *deadlines* en orden creciente.
2. Recorrer las tareas en este orden y ejecutar cada tarea si es posible completarla antes de su *deadline*.

Pseudocódigo

```
Algoritmo: MaxTareas(T, D)
Entrada: T = conjunto de n tareas
D = vector de deadlines d[1..n]
Salida: Número máximo de tareas que se pueden completar

1.  Ordenar las tareas por deadline: d[i1] <= d[i2] <= ... <= d[in]
2.  S <- {}  // Conjunto de tareas seleccionadas
3.  t_actual <- 0  // Tiempo actual
4.
5.  Para cada tarea t_j en orden de deadline creciente hacer:
6.      Si t_actual + 1 <= d[j] entonces:
7.          S <- S U {t_j}
8.          t_actual <- t_actual + 1
```

```
9.      Fin Si
10. Fin Para
11.
12. Retornar |S|
```

Complejidad temporal: $O(n \log n)$ debido al ordenamiento inicial.

Ejemplo de ejecución

Consideremos el ejemplo con 4 tareas y *deadlines* $D = \{2, 1, 3, 2\}$:

Iteración	Tarea	Deadline	Tiempo actual
Inicial	-	-	0
1	t_2	1	$0 \to 1$
2	t_1	2	$1 \to 2$
3	t_4	2	No factible $(2 + 1 > 2)$
4	t_3	3	$2 \to 3$

Solución: $\{t_2, t_1, t_3\}$ con 3 tareas completadas.

Demostración de correctitud

Planteamiento de la Demostración

Queremos demostrar que la solución S devuelta por el algoritmo es óptima (maximiza la cantidad de tareas).

Definición de las Soluciones

Sean las tareas de cada conjunto ordenadas cronológicamente por su tiempo de ejecución:

- Solución Greedy: $S = \langle g_1, g_2, \ldots, g_k \rangle$
- Solución Óptima: $O = \langle o_1, o_2, \ldots, o_m \rangle$

Queremos probar que $|S| = |O|$, es decir, que $k = m$.
Estrategia: Argumento por Intercambio.
Demostraremos que podemos transformar la solución óptima O en nuestra solución greedy S, paso a paso, sin que ninguna tarea incumpla su *deadline* y sin disminuir el número total de tareas.

El Paso de Intercambio

Supongamos que S y O coinciden en las primeras r posiciones, pero difieren en la posición $r+1$:

$$g_1 = o_1, \quad \ldots \quad g_r = o_r \quad \text{pero} \quad g_{r+1} \neq o_{r+1}$$

En el intervalo de tiempo $[r, r+1]$, la solución óptima ejecuta o_{r+1}, mientras que el algoritmo greedy ejecuta g_{r+1}.

- Por diseño, el algoritmo greedy elige en este paso la tarea g_{r+1} por ser una de las que tiene el **menor deadline** entre todas las tareas que aún no se han ejecutado y son factibles en el tiempo $r+1$.

- Como la solución óptima programó a o_{r+1} en este mismo instante $r+1$ y es válida, o_{r+1} también era una candidata factible.

Propiedad Greedy

Como el algoritmo greedy eligió a g_{r+1} teniendo a o_{r+1} como alternativa factible, se garantiza que:

$$d(g_{r+1}) \leq d(o_{r+1})$$

Validez de la Nueva Solución

Modificamos la solución óptima O para que coincida con la elección greedy. Como **todas las tareas duran exactamente 1 unidad de tiempo**, reemplazar una tarea por otra no desplaza temporalmente al resto de la planificación. Las tareas anteriores a r siguen terminando a tiempo, y los bloques de tiempo posteriores a $r+1$ permanecen intactos.

Solo debemos asegurar que los *deadlines* de las tareas intercambiadas se sigan cumpliendo. Tenemos dos casos:

Caso 1: g_{r+1} no está en O
Reemplazamos o_{r+1} por g_{r+1} en la posición temporal $r+1$. Como g_{r+1} fue la elección greedy en ese preciso instante, sabemos que es factible ($r+1 \leq d(g_{r+1})$). La nueva solución es válida.

Caso 2: g_{r+1} ya está en O, programada más adelante (en un instante $j > r+1$)
Hacemos un intercambio: ponemos g_{r+1} en el tiempo $r+1$, y movemos o_{r+1} al tiempo j.

- La tarea g_{r+1} pasa a ejecutarse antes ($r+1 < j$), por lo que cumple con su deadline con un mejor margen de tiempo.

- La tarea o_{r+1} pasa a ejecutarse en j. ¿Sigue cumpliendo su deadline? Sabemos que en la planificación O original, la tarea g_{r+1} era válida en j, es decir, $j \leq d(g_{r+1})$. Utilizando la Propiedad Greedy ($d(g_{r+1}) \leq d(o_{r+1})$), obtenemos por transitividad:

  $$j \leq d(g_{r+1}) \leq d(o_{r+1})$$

  ¡Por lo tanto, o_{r+1} también cumple su *deadline* en su nueva posición más tardía!

En ambos casos, la nueva solución modificada O' es completamente factible, mantiene la misma cantidad de tareas completadas que O (sigue siendo óptima), y ahora coincide con las decisiones greedy en un paso adicional.

Conclusión de la demostración

Aplicando este argumento de intercambio de forma inductiva, podemos sustituir o reordenar consecutivamente todos los elementos de la solución óptima hasta que O se transforme en S **sin modificar la cantidad de tareas completadas**. Por lo tanto, se demuestra que la estrategia greedy siempre garantiza la cantidad óptima de tareas.

Ejercicio 6. Producto Escalar

Enunciado del problema

Dados dos vectores $v, w \in \mathbb{R}^n$, queremos encontrar permutaciones σ y τ de sus coordenadas que minimicen el producto escalar (también conocido como producto interno) entre los vectores reordenados.

Entrada:

- Dos vectores $v = (v_1, v_2, \ldots, v_n)$ y $w = (w_1, w_2, \ldots, w_n)$ en $\mathbb{R}^n$

Salida:

- Permutaciones $\sigma, \tau : \{1, \ldots, n\} \to \{1, \ldots, n\}$ tales que el producto escalar

$$\langle v_\sigma, w_\tau \rangle = \sum_{i=1}^{n} v_{\sigma(i)} \cdot w_{\tau(i)}$$

sea mínimo, donde $v_\sigma = (v_{\sigma(1)}, \ldots, v_{\sigma(n)})$ y $w_\tau = (w_{\tau(1)}, \ldots, w_{\tau(n)})$

Objetivo: Diseñar un algoritmo eficiente que encuentre las permutaciones óptimas y demostrar su correctitud.

Algoritmo Greedy propuesto

Estrategia: Ordenar un vector de forma creciente y el otro de forma decreciente, luego emparejar las coordenadas en ese orden.

Algoritmo

```
Algoritmo: MinimizarProductoEscalar(v, w)
Entrada: Vectores v, w en R^n
Salida: las permutaciones sigma, tau que minimizan el producto escalar

1. sigma <- ObtenerÍndicesOrdenados(v, creciente)
2. tau <- ObtenerÍndicesOrdenados(w, decreciente)
3. Retornar (sigma, tau)
```

Complejidad: $O(n \log n)$ debido a los ordenamientos.

Ejemplo

Sean $v = (3, 1, 4, 2)$ y $w = (5, 2, 8, 1)$.

Con el orden propuesto:

$$\langle v_{\text{ord}}, w_{\text{ord}} \rangle = 1 \cdot 8 + 2 \cdot 5 + 3 \cdot 2 + 4 \cdot 1 = 8 + 10 + 6 + 4 = 28$$

Con el orden original:

$$\langle v, w \rangle = 3 \cdot 5 + 1 \cdot 2 + 4 \cdot 8 + 2 \cdot 1 = 15 + 2 + 32 + 2 = 51$$

El algoritmo greedy produce efectivamente un valor menor (28 vs 51). **Esto no demuestra la correctitud**, pero sirve como testeo inicial.

Demostración de correctitud

Planteamiento de la Demostración

Sean $v = (v_1, \ldots, v_n)$ y $w = (w_1, \ldots, w_n)$ dos vectores en $\mathbb{R}^n$. Queremos demostrar que el producto escalar se minimiza cuando ordenamos las coordenadas de v en orden creciente y las de w en orden decreciente.

Notar que este no es el único orden posible que minimiza el producto. Por ejemplo, ordenar v en orden decreciente y w en orden creciento nos daría el mismo valor. Pensar qué tienen estos órdenes en común nos ayudará a pensar la demostración:

Lema (invariancia a intercambios conjuntos). *Sean σ y τ dos permutaciones válidas. Sea σ' y τ' dos permutaciones válidas tales que solamente solamente difieren de σ y τ en dos posiciones, es decir, existen i, j tales que $\sigma(k) = \sigma'(k)$ y $\tau(k) = \tau'(k)$ para todo $k \neq i, j$, pero $\sigma(i) = \sigma'(j), \sigma(j) = \sigma'(i), \tau(i) = \tau'(j), \tau(j) = \tau'(i)$. Entonces, $\sum_{k=1}^{n} v_{\sigma(k)} \cdot w_{\tau(k)} = \sum_{k=1}^{n} v_{\sigma'(k)} \cdot w_{\tau'(k)}$.*

En criollo, si intercambiamos en las dos secuencias (v y w) de la misma manera, el producto escalar es el mismo.

Demostración: Esto se cumple ya que

$$\begin{aligned}
\sum_{k=1}^{n} v_{\sigma(k)} \cdot w_{\tau(k)} &= \sum_{k=1, k\neq i,j}^{n} v_{\sigma(k)} \cdot w_{\tau(k)} + v_{\sigma(i)} \cdot w_{\tau(i)} + v_{\sigma(j)} \cdot w_{\tau(j)} \\
&= \sum_{k=1, k\neq i,j}^{n} v_{\sigma'(k)} \cdot w_{\tau'(k)} + v_{\sigma'(j)} \cdot w_{\tau'(j)} + v_{\sigma'(i)} \cdot w_{\tau'(i)} \\
&= \sum_{k=1}^{n} v_{\sigma'(k)} \cdot w_{\tau'(k)} \quad \square
\end{aligned}$$

Esta propiedad nos dice algo muy importante: lo que importa para el valor del producto escalar es el orden relativo de los elementos de v con respecto a los de w. Esto nos permite fijar el orden de uno de los dos y ver qué debe pasar con el otro para minimizar el valor.

Definición de las Soluciones

Sean las siguientes permutaciones:

- Solución Greedy: $S = (\{v_1, v_2, \ldots, v_n\}, \{w_1, w_2, \ldots, w_n\})$
- Solución Óptima: $O = (\{\hat{v}_1, \hat{v}_2, \ldots, \hat{v}_n\}, \{\hat{w}_1, \hat{w}_2, \ldots, \hat{w}_n\})$

Trabajaremos directamente con los valores, y no con los σ y τ para simplificar la notación Sabemos que $v_1 \leq v_2 \leq \cdots \leq v_n$ y $w_1 \geq w_2 \geq \cdots \geq w_n$ por definición de nuestra estrategia greedy. Asumimos, sin pérdida de la generalidad (por el lema anterior), que $\hat{v}_1 \leq \hat{v}_2 \leq \cdots \leq \hat{v}_n$. Por lo tanto, $v = \hat{v}$ y las diferencias entre S y O solo pueden estar en w y $\hat{w}$.

Para ver que greedy es óptimo, demostraremos que $\sum_{k=1}^{n} v_k \cdot w_k = \sum_{k=1}^{n} \hat{v}_k \cdot \hat{w}_k$. Estrategia: Argumento por Intercambio. Demostraremos que podemos transformar la solución óptima O en nuestra solución greedy S, intercambiando elementos en $\hat{w}$.

El Paso de Intercambio

Supongamos que w y $\hat{w}$ coinciden en los primeros k valores, pero difieren en el valor $k+1$:

$$w_1 = \hat{w}_1, \quad w_2 = \hat{w}_2, \quad \ldots \quad w_r = \hat{w}_r \quad \text{pero} \quad w_{r+1} \neq \hat{w}_{r+1}$$

Como los valores de w son los mismos que los de $\hat{w}$ (solo cambia el orden), w_{r+1} debe estar en alguna otra posición siguiente de $\hat{w}$. Es decir, existe $j > r+1$ tal que $w_{r+1} = \hat{w}_j$. Notar además que, como la estrategia greedy ordena a w de manera decreciente, debe ocurrir que $w_{r+1} > \hat{w}_{r+1}$.

Definimos la solución $O' = (\hat{v}', \hat{w}')$ tomando O e intercambiando (swap) $\hat{w}_{r+1}$ por $\hat{w}_j$. **La solución O' sigue siendo válida** ya que es un reordenamiento de los mismos elementos.

Veamos la diferencia entre O y O' del valor del producto escalar:

$$\sum_{k=1}^{n} \hat{v}'_k \cdot \hat{w}'_k - \sum_{k=1}^{n} \hat{v}_k \cdot \hat{w}_k = \hat{v}'_{r+1} \cdot \hat{w}'_{r+1} + \hat{v}'_j \cdot \hat{w}'_j - \hat{v}_{r+1} \cdot \hat{w}_{r+1} - \hat{v}_j \cdot \hat{w}_j$$

Notar que las variables pintadas de un mismo color valen lo mismo (por el intercambio). Además, $w_{r+1} = \hat{w}_j$. Luego, reemplazamos:

$$\sum_{k=1}^{n} \hat{v}'_k \cdot \hat{w}'_k - \sum_{k=1}^{n} \hat{v}_k \cdot \hat{w}_k = \hat{v}'_{r+1} \cdot w_{r+1} + \hat{v}'_j \cdot \hat{w}_{r+1} - \hat{v}_{r+1} \cdot \hat{w}_{r+1} - \hat{v}_j \cdot w_{r+1}$$

Sacamos factor común:

$$\sum_{k=1}^{n} \hat{v}'_k \cdot \hat{w}'_k - \sum_{k=1}^{n} \hat{v}_k \cdot \hat{w}_k = (\hat{v}'_{r+1} - \hat{v}_j) \cdot w_{r+1} + (\hat{v}'_j - \hat{v}_{r+1}) \cdot \hat{w}_{r+1}$$

Usando que $\hat{v} = \hat{v}'$ (el intercambio no modificó $\hat{v}$), tenemos:

$$\begin{aligned} \sum_{k=1}^{n} \hat{v}'_k \cdot \hat{w}'_k - \sum_{k=1}^{n} \hat{v}_k \cdot \hat{w}_k &= (\hat{v}'_{r+1} - \hat{v}_j) \cdot w_{r+1} - (\hat{v}_{r+1} - \hat{v}'_j) \cdot \hat{w}_{r+1} \\ &= (\hat{v}_{r+1} - \hat{v}_j) \cdot (w_{r+1} - \hat{w}_{r+1}) \end{aligned}$$

Como $\hat{v}$ estaba ordenado crecientemente y $r+1 < j$, $\hat{v}_{r+1} - \hat{v}_j \leq 0$. Además, sabemos que $w_{r+1} > \hat{w}_{r+1}$, por lo que $w_{r+1} - \hat{w}_{r+1} > 0$. Luego,

$$\sum_{k=1}^{n} \hat{v}'_k \cdot \hat{w}'_k - \sum_{k=1}^{n} \hat{v}_k \cdot \hat{w}_k = (\hat{v}_{r+1} - \hat{v}_j) \cdot (w_{r+1} - \hat{w}_{r+1}) \leq 0$$
$$\sum_{k=1}^{n} \hat{v}'_k \cdot \hat{w}'_k - \sum_{k=1}^{n} \hat{v}_k \cdot \hat{w}_k \leq 0$$

Es decir, nuestra solución O' será mejor o igual que O. Como O es óptima, debe ocurrir que el valor de O' es igual al de O.

Conclusión de la demostración

Aplicando este argumento de forma inductiva, podemos intercambiar consecutivamente todos los elementos de la solución óptima hasta que O se transforme en S **sin aumentar el valor del producto escalar**. Por lo tanto, $\sum_{k=1}^{n} v_k \cdot w_k = \sum_{k=1}^{n} \hat{v}_k \cdot \hat{w}_k$. Es decir, el algoritmo greedy minimiza el producto escalar.

Observación importante

Que el intercambio (swap) no aumente el valor del producto escalar solo puede ocurrir si $\hat{v}_{r+1} = \hat{v}_j$. Esto nos indica que podemos tener múltiples soluciones óptimas si tenemos números repetidos. Si el enunciado indicara que no hay números repetidos, en la demostración veríamos que O' es estrictamente mejor que O. Esto es un absurdo, ya que dijimos que O es óptimo. Este absurdo surgiría de que asumimos que $w \neq \hat{w}$, por lo tanto demostrando que $S = O$.

Aunque no es el caso en este ejercicio, **es algo que va a pasar siempre que la solución sea única**. Por eso **es importante entender bien el problema y las soluciones óptimas** para demostrar correctamente la optimalidad.